

Punto 4.6 - Over-fetching: causas y soluciones

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el over-fetching y las soluciones. Comparar el volumen de datos y el tiempo de ejecución. Analizar el SQL generado por cada técnica. Aplicar proyecciones, filtros y paginación.

Contexto del proyecto: En el punto 4.5 se estudiaron las soluciones al problema N+1: Include, ThenInclude, proyecciones y AsSplitQuery. En este punto se profundiza en el over-fetching, que es el problema de cargar más datos de los necesarios. Esta técnica se usará en el punto 4.7 para las consultas ineficientes.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de over-fetching a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerOrdenesCompletas();
    List<OrdenResumenDto> ObtenerResumenesProyectados();
    List<OrdenResumenDto> ObtenerResumenesPaginados(int pagina, int tamanoPagina);
    List<OrdenResumenDto> ObtenerResumenesFiltradosYProyectados(string estado); void
    Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerOrdenesCompletas(); → método que carga las órdenes completas.

Línea 11: List ObtenerResumenesProyectados(); → método que carga los resúmenes con proyección.

Línea 12: List ObtenerResumenesPaginados(int pagina, int tamanoPagina); → método que carga los resúmenes con paginación.

Línea 13: List ObtenerResumenesFiltradosYProyectados(string estado); → método que carga los resúmenes con filtro y proyección.

Línea 15: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

■ Paso 3: Implementar los métodos de over-fetching en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<OrdenFabricacion> ObtenerOrdenesCompletas()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenResumenDto> ObtenerResumenesProyectados()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenResumenDto> ObtenerResumenesPaginados(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenResumenDto> ObtenerResumenesFiltradosYProyectados(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

Línea 1: `public List ObtenerOrdenesCompletas()` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 6: `.ToList();` → materializa la consulta con todas las columnas.

Línea 9: `public List ObtenerResumenesProyectados()` → declara el método.

Línea 11: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 12: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 13: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 14: `.Select(o => new OrdenResumenDto)` → proyecta al DTO.

Línea 15: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 16: `Cliente = o.Cliente,` → asigna el cliente.

Línea 17: `Estado = o.Estado,` → asigna el estado.

Línea 18: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 19: `}` → cierra la proyección.

Línea 20: `.ToList();` → materializa la consulta solo con las columnas proyectadas.

Línea 23: `public List ObtenerResumenesPaginados(int pagina, int tamanoPagina)` → declara el método.

Línea 25: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 26: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 27: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 28: `.Skip((pagina - 1) * tamanoPagina)` → salta las páginas anteriores.

Línea 29: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 30: `.Select(o => new OrdenResumenDto)` → proyecta al DTO.

Línea 31: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 32: `Cliente = o.Cliente,` → asigna el cliente.

Línea 33: `Estado = o.Estado,` → asigna el estado.

Línea 34: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 35: `}` → cierra la proyección.

Línea 36: `.ToList();` → materializa la consulta con paginación y proyección.

Línea 39: `public List ObtenerResumenesFiltradosYProyectados(string estado)` → declara el método.

Línea 41: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 42: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 43: `.Where(o => o.Estado == estado)` → filtra por estado.

Línea 44: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 45: `.Select(o => new OrdenResumenDto)` → proyecta al DTO.

Línea 46: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 47: `Cliente = o.Cliente,` → asigna el cliente.

Línea 48: `Estado = o.Estado,` → asigna el estado.

Línea 49: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 50: `}` → cierra la proyección.

Línea 51: .ToList(); → materializa la consulta con filtro y proyección.

Error común: si se aplica Skip sin OrderBy, el resultado es indeterminado. Se debe aplicar OrderBy antes de Skip.

▮ Paso 4: Crear el caso de uso de over-fetching

Crear el archivo src/AceriaData.Application/UseCases/OverFetchingUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class OverFetchingUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public OverFetchingUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== OVER-FETCHING ===");

        DemostrarEntidadesCompletas();
        DemostrarProyeccion();
        DemostrarPaginacion();
        DemostrarFiltroYProyeccion();
        CompararRendimiento();
    }

    private void DemostrarEntidadesCompletas()
    {
        Console.WriteLine("\n--- Entidades completas (over-fetching) ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad Ordenes.ObtenerOrdenesCompletas();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        }
    }

    private void DemostrarProyeccion()
    {
        Console.WriteLine("\n--- Proyección (sin over-fetching de columnas) ---");

        var cronometro = Stopwatch.StartNew();
        var resúmenes = _unidad Ordenes.ObtenerResúmenesProyectados();
        cronometro.Stop();

        Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var resumen in resúmenes)
        {
            Console.WriteLine($"  {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");
        }
    }

    private void DemostrarPaginacion()
    {
        Console.WriteLine("\n--- Paginación (sin over-fetching de filas) ---");

        var cronometro = Stopwatch.StartNew();
        var resúmenes = _unidad Ordenes.ObtenerResúmenesPaginados(1, 3);
        cronometro.Stop();
    }
}
```

```

ms");
    Console.WriteLine($"Resúmenes: {resumenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds}");
    foreach (var resumen in resumenes)
    {
        Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");
    }
}

private void DemostrarFiltroYProyeccion()
{
    Console.WriteLine("\n--- Filtro y proyección ---");

    var cronometro = Stopwatch.StartNew();
    var resumenes = _unidad Ordenes.ObtenerResumenesFiltradosYProyectados("Pendiente");
    cronometro.Stop();

    Console.WriteLine($"Resúmenes pendientes: {resumenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    foreach (var resumen in resumenes)
    {
        Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");
    }
}

private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroCompletas = Stopwatch.StartNew();
    var ordenesCompletas = _unidad Ordenes.ObtenerOrdenesCompletas();
    cronometroCompletas.Stop();

    var cronometroProyectadas = Stopwatch.StartNew();
    var resumenesProyectados = _unidad Ordenes.ObtenerResumenesProyectados();
    cronometroProyectadas.Stop();

    Console.WriteLine($"Entidades completas: {cronometroCompletas.ElapsedMilliseconds} ms | Registros: {ordenesCompletas.Count}");
    Console.WriteLine($"Proyección: {cronometroProyectadas.ElapsedMilliseconds} ms | Registros: {resumenesProyectados.Count}");
}
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class OverFetchingUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public OverFetchingUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== OVER-FETCHING ==="); → muestra la cabecera.

Línea 19: DemostrarEntidadesCompletas(); → llama al método de entidades completas.

Línea 20: DemostrarProyeccion(); → llama al método de proyección.

Línea 21: DemostrarPaginacion(); → llama al método de paginación.

Línea 22: DemostrarFiltroYProyeccion(); → llama al método de filtro y proyección.

Línea 23: CompararRendimiento(); → llama al método de comparación.

Línea 26: `private void DemostrarEntidadesCompletas()` → declara el método.

Línea 28: `Console.WriteLine("\n--- Entidades completas (over-fetching) ---");` → muestra la cabecera.

Línea 30: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 31: `var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletas();` → carga las órdenes completas.

Línea 32: `cronometro.Stop();` → detiene el cronómetro.

Línea 34: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de órdenes y el tiempo.

Línea 35: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 37: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos.

Línea 40: `private void DemostrarProyeccion()` → declara el método.

Línea 42: `Console.WriteLine("\n--- Proyección (sin over-fetching de columnas) ---");` → muestra la cabecera.

Línea 44: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 45: `var resúmenes = _unidad.Ordenes.ObtenerResúmenesProyectados();` → carga los resúmenes.

Línea 46: `cronometro.Stop();` → detiene el cronómetro.

Línea 48: `Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 49: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 51: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");` → muestra los datos.

Línea 54: `private void DemostrarPaginacion()` → declara el método.

Línea 56: `Console.WriteLine("\n--- Paginación (sin over-fetching de filas) ---");` → muestra la cabecera.

Línea 58: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 59: `var resúmenes = _unidad.Ordenes.ObtenerResúmenesPaginados(1, 3);` → carga los resúmenes paginados.

Línea 60: `cronometro.Stop();` → detiene el cronómetro.

Línea 62: `Console.WriteLine($"Resúmenes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 63: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 65: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");` → muestra los datos.

Línea 68: `private void DemostrarFiltroYProyeccion()` → declara el método.

Línea 70: `Console.WriteLine("\n--- Filtro y proyección ---");` → muestra la cabecera.

Línea 72: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 73: `var resúmenes = _unidad.Ordenes.ObtenerResúmenesFiltradosYProyectados("Pendiente");` → carga los resúmenes filtrados y proyectados.

Línea 74: `cronometro.Stop();` → detiene el cronómetro.

Línea 76: `Console.WriteLine($"Resúmenes pendientes: {resúmenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el número de resúmenes y el tiempo.

Línea 77: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 79: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado}");` → muestra los datos.

Línea 83: `private void CompararRendimiento()` → declara el método.

Línea 85: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 87: `var cronometroCompletas = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 88: `var ordenesCompletas = _unidad Ordenes.ObtenerOrdenesCompletas();` → carga las órdenes completas.

Línea 89: `cronometroCompletas.Stop();` → detiene el cronómetro.

Línea 91: `var cronometroProyectadas = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 92: `var resúmenesProyectados = _unidad Ordenes.ObtenerResúmenesProyectados();` → carga los resúmenes proyectados.

Línea 93: `cronometroProyectadas.Stop();` → detiene el cronómetro.

Línea 95: `Console.WriteLine($"Entidades completas: {cronometroCompletas.ElapsedMilliseconds} ms | Registros: {ordenesCompletas.Count}");` → muestra el tiempo y el número de registros.

Línea 96: `Console.WriteLine($"Proyección: {cronometroProyectadas.ElapsedMilliseconds} ms | Registros: {resúmenesProyectados.Count}");` → muestra el tiempo y el número de registros.

Error común: si se proyecta a un DTO con propiedades no asignadas, EF Core deja las propiedades con su valor por defecto. Se deben asignar todas las propiedades.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<OverFetchingUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<OverFetchingUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado =
        "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado =
        "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0007", Cliente = "Constructora del Este", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 7, 22) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0008", Cliente = "Constructora del Oeste", Estado =
        "Pendiente", FechaCreacion = new DateTime(2024, 8, 30) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... }, → sexta orden.

Línea 15: new OrdenFabricacion { ... }, → séptima orden.

Línea 16: new OrdenFabricacion { ... } → octava orden.

Línea 17: }; → cierra la lista.

Línea 19: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 20: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

| Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de cada técnica y el SQL generado.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== OVER-FETCHING ===

--- Entidades completas (over-fetching) ---
Órdenes: 8 | Tiempo: 52 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Proyección (sin over-fetching de columnas) ---
Resúmenes: 8 | Tiempo: 40 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Paginación (sin over-fetching de filas) ---
Resúmenes: 3 | Tiempo: 38 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Filtro y proyección ---
Resúmenes pendientes: 5 | Tiempo: 39 ms
  OF-2024-0001 | Constructora del Norte | Pendiente
  ...

--- Comparación de rendimiento ---
Entidades completas: 52 ms | Registros: 8
Proyección: 40 ms | Registros: 8
```

La primera sección muestra el tiempo de cargar entidades completas. La segunda sección muestra el tiempo de cargar la proyección. La tercera sección muestra el tiempo de cargar la paginación. La cuarta sección muestra el tiempo de cargar el filtro y la proyección. La quinta sección muestra la comparación.

Observaciones: la proyección es más rápida que la carga de entidades completas. La paginación carga solo tres registros. El filtro y la proyección cargan cinco registros. En tablas con muchas filas y columnas, las diferencias son mayores.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerResúmenesPaginados para aplicar ToList antes del Skip y Take:

```
public List<OrdenResumenDto> ObtenerResúmenesPaginados(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList()
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

Resultado esperado: el código funciona, pero la consulta SQL carga todas las órdenes en memoria y después aplica Skip y Take en memoria. El over-fetching de filas no se resuelve porque todas las filas se transfieren.

Solución: aplicar Skip y Take antes de ToList.

```
public List<OrdenResumenDto> ObtenerResumenesPaginados(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToListAsync();
}
```

Resultado esperado con la solución: el SQL incluye OFFSET y FETCH y solo se transfieren las filas de la página.

Errores comunes del ejercicio

Error	Causa	Solución
Over-fetching de columnas	Se cargan entidades completas	Usar proyecciones
Over-fetching de filas	Se cargan todas las filas	Usar filtros y paginación
ToList antes de Skip	Se materializa antes de paginar	Aplicar Skip y Take antes de ToList
Include innecesario	Se incluyen entidades relacionadas que no se usan	Usar proyecciones o quitar el Include
Columnas nvarchar(max)	Se cargan columnas sin límite de longitud	Configurar longitud máxima o proyectar
Proyección incompleta	Se olvidan propiedades del DTO	Asignar todas las propiedades

Reto resuelto: Comparar el volumen de datos de entidades completas vs proyección

Reto: Crear dos métodos en el repositorio que carguen las órdenes, uno con entidades completas y otro con proyección. Comparar el volumen de datos transferidos usando el SQL generado con ToQueryString.

Solución paso a paso:

Paso 1: Añadir los métodos a la interfaz IOrdenRepositorio:

```
string ObtenerSqlEntidadesCompletas();
string ObtenerSqlProyeccionResumen();
```

Línea 1: string ObtenerSqlEntidadesCompletas(); → declara el método.

Línea 2: string ObtenerSqlProyeccionResumen(); → declara el método.

Paso 2: Implementar los métodos en OrdenRepositorio:

```
public string ObtenerSqlEntidadesCompletas()
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden);

    return consulta.ToQueryString();
}

public string ObtenerSqlProyeccionResumen()
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        });

    return consulta.ToQueryString();
}
```

Línea 1: public string ObtenerSqlEntidadesCompletas() → declara el método.

Línea 3: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.NumeroOrden); → ordena por número de orden.

Línea 7: return consulta.ToQueryString(); → devuelve el SQL.

Línea 10: public string ObtenerSqlProyeccionResumen() → declara el método.

Línea 12: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 13: .AsNoTracking() → aplica AsNoTracking.

Línea 14: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 15: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 16: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 17: Cliente = o.Cliente, → asigna el cliente.

Línea 18: Estado = o.Estado, → asigna el estado.

Línea 19: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 20: }); → cierra la proyección.

Línea 22: return consulta.ToQueryString(); → devuelve el SQL.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarSqlComparativo()
{
    Console.WriteLine("\n--- SQL de entidades completas ---");
    var sqlCompletas = _unidad Ordenes.ObtenerSqlEntidadesCompletas();
    Console.WriteLine(sqlCompletas);

    Console.WriteLine("\n--- SQL de proyección ---");
    var sqlProyeccion = _unidad Ordenes.ObtenerSqlProyeccionResumen();
    Console.WriteLine(sqlProyeccion);
}
```

Línea 1: `private void DemostrarSqlComparativo()` → declara el método.

Línea 3: `Console.WriteLine("\n--- SQL de entidades completas ---");` → muestra la cabecera.

Línea 4: `var sqlCompletas = _unidad Ordenes.ObtenerSqlEntidadesCompletas();` → obtiene el SQL.

Línea 5: `Console.WriteLine(sqlCompletas);` → imprime el SQL.

Línea 7: `Console.WriteLine("\n--- SQL de proyección ---");` → muestra la cabecera.

Línea 8: `var sqlProyeccion = _unidad Ordenes.ObtenerSqlProyeccionResumen();` → obtiene el SQL.

Línea 9: `Console.WriteLine(sqlProyeccion);` → imprime el SQL.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarSqlComparativo();
```

Paso 5: Ejecutar dotnet run y comparar el SQL.

Resultado esperado: el SQL de entidades completas incluye todas las columnas (Id, NumeroOrden, Cliente, FechaCreacion, Estado, Observaciones, IsDeleted, DeletedAt). El SQL de proyección solo incluye las columnas proyectadas (NumeroOrden, Cliente, Estado, FechaCreacion). La diferencia de volumen es notable.

Analogía final

El over-fetching en una acería es como pedir al archivo central todas las carpetas de todas las órdenes cuando solo se necesita el número de orden y el cliente de cada una. El archivo trae carpetas enteras con todos sus documentos, pero solo se usan dos datos de cada una. El over-fetching de columnas es como traer carpetas enteras cuando solo se necesitan dos datos. El over-fetching de filas es como traer todas las carpetas cuando solo se necesitan las de una página. Las proyecciones son como pedir solo los dos datos necesarios de cada carpeta. La paginación es como pedir solo las carpetas de una página. El filtro es como pedir solo las carpetas de un estado concreto. La combinación de proyección, filtro y paginación es como pedir solo los datos necesarios de las carpetas necesarias. Así funciona el over-fetching en EF Core: se identifica con el SQL generado, se mide con cronómetros y se resuelve con proyecciones, filtros y paginación.

Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de over-fetching a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `OverFetchingUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de entidades completas, proyección, paginación y filtro con proyección.

Comparado el rendimiento de cada técnica.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de aplicar ToList antes de Skip y Take.

Creado los métodos ObtenerSqlEntidadesCompletas y ObtenerSqlProyeccionResumen.
